A fordítóprogramok általánosan forrásnyelvi szövegből állítanak elő tárgykódot. A forrásprogram beolvasása után elvégzik a lexikális, szintaktikus és szemantikus elemzést, előállítják a szintaxis fát, generálják, majd optimalizálják a tárgykódot.

Szintaktikus elemzés: A szintaktikus elemzőnek a feladata a program struktúrájának a felismerése. A szintaktikus elemző működésének az eredménye lehet például az elemzett program szintaxisfája vagy ezzel ekvivalens struktúra. Bemenete egy szimbólumsorozat, eredménye pedig a szintaktikusan elemzett program, és ha vannak, akkor a szintaktikai hibák. A ~ környezetfüggetlen grammatikával leírható tulajdonságok vizsgálata.
(a környezetfüggetlen nyelvtan egy formális nyelvtan, amelyben minden produkciós szabály a következő formájú: V → w, ahol V egy nem-terminális szimbólum és w egy jelsorozat, amely terminális és/vagy nem-terminális szimbólumokat tartalmaz. A nem-terminális V minden esetben helyettesíthető w-vel, függetlenül attól, hogy milyen környezetben fordul V elő. Egy formális nyelv akkor környezetfüggetlen, ha környezetfüggetlen nyelvtan generálja.)

Lexikális elemző: alapvető feladata az, hogy a forrásnyelvű program lexikális egységeit felismerje, azaz meghatározza a forrásnyelvű kódban a szimbólumok szövegét és típusát.
LL(k) grammatika: (név: **L**eft to right, with **L**eftmost derivation)(balról jobbra, legbal levezetéssel) A levezetés tetszőleges pontján a szöveg következő k terminálisa meghatározza az alkalmazandó levezetési szabályt. A felülről lefelé elemzések úgy működnek, hogy a nyelvtan kezdőszimbólumából kiindulva legbaloldalibb helyettesítéseket alkalmazva próbál eljutni az elemezendő szöveghez. Az LL elemzések ezen belül visszalépés nélkül oldják meg a feladatot. Az LL(k) elemzések, amiatt, hogy ne kelljen visszalépést végrehajtaniuk, k szimbólumot olvasnak előre.
LR elemzések: Alulról felfelé elemzéseknél a terminális szimbólumokból kiindulva haladnak a kezdőszimbólum felé. Ezek az elemzések egy vermet használnak, és a vizsgált módszerek léptetés-redukálás típusúak, azaz az elemző mindig egy redukciót próbál végezni, ha ez nem sikerül, akkor az input szimbólumsorozat következő elemét lépteti. Ezek az elemzések a legjobboldalibb levezetés "inverzét" adják. Az elemzés alapproblémája a mondatformák nyelének a meghatározása. Első lépésben a grammatika szabályaiból egy táblagenerátorral elkészítünk egy elemző táblázatot, majd az elemző program a táblázattal és egy veremmel tudja elemezni a szöveget.

Szintaxisfa: Legyen G=(N,T,P,S) tetszőleges környezetfüggetlen grammatika. G-beli szintaxis (levezetési) fának nevezünk minden olyan fagráfot, amelynek a csúcsait az N ∪ T halmaz elemei jelölik. Egy mondat szintaxisfája:
- A gyökérhez az S kezdőszimbólum tartozik
- A levelei a grammatika terminális szimbólumai
- A többi pontja a nemterminális szimbólumok halmaza

NetBeans: az elnevezés egyaránt jelöl egy platform-ot, és egy IDE-t. A NetBeans Platform egy újrafelhasználható keretrendszer a programfejlesztés egyszerűsítésére. Moduláris felépítésű, elérhető funkcióit az aktuálisan betöltött modulok határozzák meg. A modulok dinamikusan (is) betölthetők, frissíthetők. A platform számos szolgáltatást nyújt a fejlesztőknek: háttértár kezelés (pl.: SQL), ablakkezelés, varázsló keretrendszer stb. A NetBeans IDE egy Java nyelven megírt fejlesztői környezet, ami a NetBeans Platformot használja. Moduláris rendszer, minden szolgáltatását egy modul adja, így a szolgáltatásai bővíthetőek.

NetBeans Java technológiája: lehetővé teszi bármilyen típusú (J2SE, J2EE, web, mobile) Java program készítését. Lehetőség van desktop alkalmazások készítésére, ehhez rendelkezésre áll GUI készítő eszköz, aminek segítségével gyorsan, grafikusan lehet összeállítani GUI felületeket, és nem kell közvetlenül a kódban összeállítani ezeket.

Refaktorizáció: számos olyan eljárás, aminek segítségével a már megírt kódon tudunk változtatni, nagymértékben automatizált módon. Ilyen eljárások pl.: osztály átnevezése a hivatkozásaival együtt, paraméterhozzáadás/-törlés/ -átrendezés a hivatkozási helyeken is, osztályváltozó mozgatása ős/leszármazott osztályba, osztály másolása/ áthelye-zése másik csomagba/osztályba, biztonságos törlés (az elem és összes hivatkozásának törlése)

eXtensive Markup Language (XML): egy általános célú leíró nyelv, speciális célú leíró nyelvek létrehozására. Elsődleges célja strukturált szöveg és információ megosztása. Az adatokat egy faszerkezetben tárolja, ezen belül mindent elemek formájában tárol, különböző tagekkel jelölve. Egy elem formátuma kötött (***<elem attr1="érték1" attr2="érték2"…> tartalom </elem>***)

Simple Object Access Protocol (SOAP): egy protokoll XML alapú üzenetek cseréjére, általában http alapokon. Rendszeresen a WSDL-el (Web Services Description Language) együtt szokták alkalmazni, ami egy XML alapú leíró felületet biztosít a webszolgáltatásokhoz.

Neumann-elvek: Neumann János dolgozta ki a számítógépek ideális működéséhez.
1. soros utasítás: utasítások végrehajtása időben egymás után
2. kettes (bináris) számrendszer használata: A kettes számrendszert és a rajta értelmezett aritmetikai ill. logikai műveleteket könnyű megvalósítani kétállapotú áramkörökkel.
3. belső memória (operatív tár) használata a program és az adatok tárolására
4. teljesen elektronikus működés: számítógép központi egységében nincsenek mozgó alkatrészek, ellentétben a régi mechanikus vagy elektromechanikus számológépekkel.
5. széles körű felhasználhatóság: alkalmas bármilyen adatfeldolgozási feladatra (univerzális Turing-gépként működik)

Neumann-elvű számítógépek alapvető hardveregységei: központi egység (CU) [központi vezérlő egység (Control Unit)+ALU+regiszterblokk+cache+FPU(lebegőpontos matematikai társprocesszor)], operatív tár(memória), háttértárak, perifériák.

Neumann-ciklus: A program a RAM memóriába töltődik, majd az utasításmutató feltöltődik a program első utasításának címével. Az utasítás-végrehajtási ciklusban lehívódik az utasításmutató által kijelölt utasítás, az utasításmutató a következő utasításra lép, majd a lehívott utasítás végrehajtódik. A végrehajtás eredménye sokféle lehet, többek közt az utasításmutató értéke is megváltozhat, ekkor a programvégrehajtás nem a következő utasítással folytatódik, hanem a megváltozott utasításmutató által kijelölt utasítással (ugrás). Az utasítás végrehajtásának lehet az eredménye a program leállása is (vége).

Regiszterek: processzorok ideiglenes adattárolási céljaira szolgálnak. A belső sínrendszeren keresztül tartanak kapcsolatot a processzor más részeivel.
- *Akkumulátor-regiszter*: aritmetikai-logikai műveletek operandusait, vagyis a műveletek tárgyát képező mennyiségeket vagy azoknak az eredményeit tárolja.
- *Adatszámláló regiszter*: adatok kiolvasásakor vagy beírásakor azonosított memóriarekesz címét tárolja. Mérete függ a mikroprocesszor által címezhető memóriakapacitástól. Egyszerre több is lehet belőle a CPU-ban.
- *Utasításregiszter*: végrehajtás alatt álló programutasítást tároló regiszter.
- *Utasításszámláló regiszter*: A soron következő utasítás címét tárolja. A tartalmát a program maga is változtathatja.
- *Bázis(cím)regiszter*: operandusok címzéséhez felhasznált regiszter, amely nem általános használatú. A báziscím egy alapcím, amelyhez viszonyítva adhatjuk meg az utasításban az operandus helyét.
- *Indexregiszterek*: nem minden proc.-ban találhatók, operanduscímzést segítik elő, főleg adatsorok feldolgozásánál.
- *Állapotregiszterek, vezérlő regiszterek*: egy vagy több regiszteren belül tárolnak vezérlő és ellenőrző jeleket. Az állapotregiszter az ALU műveleti eredményeit jellemzi.
- *Veremmutató regiszter (Stack Pointer)*: A verem legfelső elemét jelöli ki.

A számítógép teljesítményének szempontjából alapvető jelentősége van annak, hogy mekkora az a szóhossz, amivel a számítógép dolgozik. A szó hosszát, amellyel a processzor dolgozik, belső szóhossznak nevezzük.
(pl. 32 bites processzorok: 4 bájtos szóhossz)